

University of Tehran
College of Engineering
School of Electrical and
Computer Engineering

Machine Learning

Assignment 2

Name
Parsa KafshduziBukani

Student ID
810102501

November 23, 2025

Contents

Table of Contents	3
List of Figures	4
List of Tables	5
Abstract	6
Question 1 — Line Search Optimization Problem	7
(a) General Optimization Problem and Roles of Search Direction and Step Length	7
(b) Descent Direction	7
(c) Common Step-Length Selection Methods	7
(d) Exact Line Search on a Quadratic Function	8
Question 2 — Support Vector Machine (SVM): Primal, La- grangian, Dual, KKT, Soft Margin, and Kernelization	9
(a) Hard-Margin SVM Primal Problem	9
(b) Lagrangian and Stationarity Conditions	9
(c) Dual Problem	10
(d) KKT Conditions, Support Vectors, and Recovery of (w, b) .	10
(e) Soft-Margin SVM: Primal, Dual, and Bounds on α_i	11
(f) Kernelization and Kernel SVM Decision Function	12
Question 3 — Bilevel Optimization	12
(a) Full Bilevel Formulation	12
(b) Solving the Follower Problem and Deriving $y^*(x)$	13
(c) Substituting $y^*(x)$ into the Leader's Objective	13
(d) Solving for the Global Minimizer (x^*, y^*)	14
(e) Regularity and Validity of Reduction	15
Question 4 — Multi-Objective Optimization: Pareto Front	15
(a) Pareto Optimality	16
(b) Weighted-Sum Scalarization	16
(c) Parametric Expression for the Pareto Front	17
(d) Epsilon-Constraint and KKT Connection	17
(e) Numerical Values for Selected Weights	18

Question 5 — Genetic Algorithm: Jigsaw Puzzle Solver	18
(a) Fit Score Between Two Pieces	18
(b) Fitness of a Full Puzzle Arrangement	19
(c) Survival of the Fittest (Selection Mechanism)	19
(d) Evolutionary Engine	19
Question 7 — Batch vs. Stochastic Gradient Descent	20
(a) Computational Load per Iteration	20
(b) Noise (Variance) in the Gradient Estimation	20
(c) Stability and Nature of Convergence	21
Question 8 — Steepest Descent Direction via First-Order Taylor Approximation	21
Proof	22
Question 9 — Pseudoinverse Solution as the Least-Squares Minimizer	23
Proof	23
Question 10 — Newton and Quasi-Newton Methods	24
(a) Newton Step Computation	24
(b) When Newton's Method Can Fail to Converge	24
(c) When the Newton Direction is Not a Descent Direction	25
(d) Main Idea of Quasi-Newton Methods	25
(e) Approximating the Hessian via the Secant Equation	25
Question 11 — Quadratic Convergence of Newton's Method	26
(a) Definition of Quadratic Convergence	26
(b) Newton Step Approximates $x^* - x_k$ Near the Minimizer	27
Question 12 — Rosenbrock Function: Gradient Descent vs. Newton's Method	28
Gradient and Hessian	28
Algorithms	28
Results: Trajectories and Convergence	28
Question 13 — Equality-Constrained Optimization via Lagrange Multipliers and Newton–Raphson	30
(a) Lagrangian	31
(b) Solve $\nabla L = 0$ for (x^*, y^*, λ^*)	31
(c) The Nonlinear System $F(w) = 0$	31
(d) Jacobian $JF(w)$	32

(e) Newton–Raphson Implementation and Convergence	32
Question 14 — Step-Size Strategies in Gradient Descent	32
(a) Step-Size Selection Methods	33
(b) Gradient Descent Implementations	33
(c) Comparison of Results	33
References	36

List of Figures

1	Trajectories of Gradient Descent and Newton's Method on the Rosenbrock function.	29
2	Convergence comparison in terms of $\log_{10} f(x_k)$ for Gradient Descent and Newton's Method.	30
3	Trajectories of GD with Backtracking and AdaGrad on the Rosenbrock function.	34
4	Convergence comparison of Backtracking GD and AdaGrad.	34

List of Tables

1	Pareto-optimal solutions and objective values for selected weights.	18
---	---	----

Abstract

This assignment explored a wide range of fundamental and advanced optimization techniques used in machine learning. Beginning with line-search methods and steepest descent, we examined how descent directions and step-size rules influence convergence. We then studied stochastic and batch gradient descent, Newton and quasi-Newton methods, and the conditions under which they succeed or fail. The behavior of first- and second-order algorithms was analyzed in detail through experiments on the Rosenbrock function, including gradient descent with fixed and adaptive step-size strategies, backtracking line search, and full-step Newton's method.

We further investigated equality-constrained optimization using Lagrange multipliers and the Newton–Raphson method, solving a nonlinear system to locate constrained minima. Finally, we implemented a full Genetic Algorithm to automatically reconstruct a jigsaw puzzle, integrating fitness design, tournament selection, mutation, and a custom kernel-based crossover tailored to the puzzle's spatial structure.

Across these tasks, we observed the contrast between linear and quadratic convergence, the effect of gradient noise, the importance of step-size selection, and the benefits of second-order information or adaptive updates. The experiments confirmed theoretical properties of the algorithms and highlighted their practical performance on challenging nonconvex problems.

Question 1 — Line Search Optimization Problem

(a) General Optimization Problem and Roles of Search Direction and Step Length

Line-search methods aim to solve optimization problems of the form

$$\min_{x \in \mathbb{R}^n} f(x).$$

Starting from a current iterate x_k , a line-search algorithm updates the point using

$$x_{k+1} = x_k + \alpha_k p_k,$$

where:

- p_k is the **search direction**: a vector that points toward a direction in which we expect the function value to decrease.
- $\alpha_k > 0$ is the **step length**: a scalar controlling how far we move along the direction p_k .

(b) Descent Direction

A vector p is called a **descent direction** for f at point x if moving a small amount along p reduces the function value. The necessary condition for p to be a descent direction is:

$$\nabla f(x)^\top p < 0.$$

(c) Common Step-Length Selection Methods

Common strategies to choose the step length α include:

- **Exact Line Search**: chooses α that minimizes $f(x + \alpha p)$ exactly along the line.
- **Backtracking / Armijo Rule**: starts from an initial α and repeatedly shrinks it until the Armijo sufficient decrease condition

$$f(x + \alpha p) \leq f(x) + c\alpha \nabla f(x)^\top p$$

is satisfied.

- **Wolfe Conditions:** chooses α such that (i) the Armijo condition holds, and (ii) the curvature condition

$$\nabla f(x + \alpha p)^\top p \geq c_2 \nabla f(x)^\top p$$

is also satisfied.

- **Fixed Step Size:** a constant α is used across iterations. Simple but may not converge.
- **Barzilai–Borwein (BB) Method:** chooses α using gradient differences to approximate curvature, e.g.,

$$\alpha_k = \frac{s_{k-1}^\top s_{k-1}}{s_{k-1}^\top y_{k-1}},$$

where $s_{k-1} = x_k - x_{k-1}$ and $y_{k-1} = \nabla f(x_k) - \nabla f(x_{k-1})$.

(d) Exact Line Search on a Quadratic Function

Consider the quadratic function

$$f(x_1, x_2) = 2x_1^2 + x_1x_2 + 4x_2^2,$$

with current iterate and search direction

$$x^{(t)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad p^{(t)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

1. Verifying the Descent Direction

The gradient of f is

$$\nabla f(x) = \begin{bmatrix} 4x_1 + x_2 \\ x_1 + 8x_2 \end{bmatrix}.$$

Evaluating at $x^{(t)}$:

$$\nabla f(x^{(t)}) = \begin{bmatrix} 1 \\ 8 \end{bmatrix}.$$

Compute the directional derivative:

$$\nabla f(x^{(t)})^\top p^{(t)} = \begin{bmatrix} 1 & 8 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} = -15 < 0.$$

Thus, $p^{(t)}$ is a descent direction.

2. Forming the One-Dimensional Function $\phi(\alpha)$

$$x^{(t)} + \alpha p^{(t)} = \begin{bmatrix} \alpha \\ 1 - 2\alpha \end{bmatrix}.$$

Hence

$$\phi(\alpha) = f(\alpha, 1 - 2\alpha) = 16\alpha^2 - 15\alpha + 4.$$

3. Exact Minimizer α^*

For a quadratic $\phi(\alpha) = a\alpha^2 + b\alpha + c$, the minimizer is

$$\alpha^* = -\frac{b}{2a} = \frac{15}{32} > 0.$$

Therefore, the exact step length is positive and valid.

Question 2 — Support Vector Machine (SVM): Primal, Lagrangian, Dual, KKT, Soft Margin, and Kernelization

(a) Hard-Margin SVM Primal Problem

For linearly separable data, the hard-margin SVM seeks the separating hyperplane $w^\top x + b = 0$ of maximum margin. The primal optimization problem is

$$\begin{array}{ll} \min_{w,b} & \frac{1}{2} \|w\|^2 \\ \text{s.t.} & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \dots, n. \end{array}$$

The objective maximizes the margin $1/\|w\|$, while the constraints enforce correct classification with margin at least 1.

(b) Lagrangian and Stationarity Conditions

We introduce Lagrange multipliers $\alpha_i \geq 0$ for each inequality constraint and write the Lagrangian:

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i(w^\top x_i + b) - 1).$$

Derivative w.r.t. w .

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0 \quad \Rightarrow \quad \boxed{w = \sum_{i=1}^n \alpha_i y_i x_i.}$$

Derivative w.r.t. b .

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0 \quad \Rightarrow \quad \boxed{\sum_{i=1}^n \alpha_i y_i = 0.}$$

These stationarity conditions will be substituted into the dual problem.

(c) Dual Problem

Substituting $w = \sum_i \alpha_i y_i x_i$ into the Lagrangian and eliminating w and b , we obtain the dual:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

The dual is a convex quadratic program in the α_i only.

(d) KKT Conditions, Support Vectors, and Recovery of (w, b)

The Karush–Kuhn–Tucker (KKT) complementary slackness condition is

$$\boxed{\alpha_i (y_i (w^\top x_i + b) - 1) = 0, \quad i = 1, \dots, n.}$$

Interpretation:

- If $\alpha_i = 0$, then $y_i (w^\top x_i + b) > 1$ and x_i lies strictly outside the margin.
- If $\alpha_i > 0$, then $y_i (w^\top x_i + b) = 1$ and x_i lies exactly on a margin boundary.

Thus:

$$\boxed{\text{Support vectors are the points with } \alpha_i > 0.}$$

The optimal primal variables are recovered by:

Recovering w .

$$w^* = \sum_{i=1}^n \alpha_i^* y_i x_i.$$

Recovering b . For any support vector x_s :

$$y_s(w^{*\top} x_s + b^*) = 1 \quad \implies \quad b^* = y_s - w^{*\top} x_s.$$

All support vectors give the same b^* (up to numerical precision).

(e) Soft-Margin SVM: Primal, Dual, and Bounds on α_i

When the data are not linearly separable, slack variables $\xi_i \geq 0$ permit controlled violations of the margin constraints.

Soft-margin primal.

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0. \end{aligned}$$

Dual problem. The dual becomes:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \\ & \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

Thus the dual multipliers are **bounded**:

$$0 \leq \alpha_i \leq C.$$

(f) Kernelization and Kernel SVM Decision Function

In the dual formulation, the data appear only through inner products $\langle x_i, x_j \rangle$. We may replace this by any kernel function

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle,$$

which implicitly maps the data to a higher-dimensional feature space.

The dual with kernels is

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j).$$

The resulting classifier is:

$$f(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i^* y_i K(x_i, x) + b^* \right).$$

This enables SVMs to learn nonlinear decision boundaries through kernel functions such as polynomial, Gaussian RBF, or sigmoid kernels.

Question 3 — Bilevel Optimization

(a) Full Bilevel Formulation

The bilevel optimization problem is written as

$$\min_{x \geq 0} F(x, y)$$

subject to

$$y \in \arg \min_{y \geq 0} f(x, y).$$

Explicitly,

$$\begin{aligned} \text{Upper level (leader):} \quad & \min_{x \geq 0} (x - 2)^2 + (y - 3)^2, \\ \text{Lower level (follower):} \quad & y \in \arg \min_{y \geq 0} (y - x^2)^2 + y. \end{aligned}$$

(b) Solving the Follower Problem and Deriving $y^*(x)$

The follower solves the constrained one-dimensional optimization

$$\min_{y \geq 0} (y - x^2)^2 + y.$$

We first determine the unconstrained minimizer. Differentiate w.r.t. y :

$$\frac{\partial f}{\partial y} = 2(y - x^2) + 1.$$

Setting the derivative to zero:

$$2(y - x^2) + 1 = 0 \implies y = x^2 - \frac{1}{2}.$$

Now apply the constraint $y \geq 0$:

$$x^2 - \frac{1}{2} \geq 0 \iff x \geq \frac{1}{\sqrt{2}}.$$

Thus the follower chooses

$$y^*(x) = \begin{cases} 0, & x < \frac{1}{\sqrt{2}}, \\ x^2 - \frac{1}{2}, & x \geq \frac{1}{\sqrt{2}}. \end{cases}$$

This piecewise behavior reflects “projection” of the unconstrained minimizer onto the feasible region.

(c) Substituting $y^*(x)$ into the Leader’s Objective

The leader’s objective is

$$F(x, y) = (x - 2)^2 + (y - 3)^2.$$

We substitute $y^*(x)$ into F to reduce the bilevel problem to a single-variable optimization.

Case 1: $x < \frac{1}{\sqrt{2}}$. Here $y^*(x) = 0$, so

$$F_1(x) = (x - 2)^2 + (0 - 3)^2 = (x - 2)^2 + 9.$$

Case 2: $x \geq \frac{1}{\sqrt{2}}$. Here $y^*(x) = x^2 - \frac{1}{2}$, so

$$F_2(x) = (x - 2)^2 + \left(x^2 - \frac{1}{2} - 3\right)^2 = (x - 2)^2 + (x^2 - 3.5)^2.$$

The upper-level optimization therefore becomes:

$$\min_{x \geq 0} \begin{cases} F_1(x), & 0 \leq x < \frac{1}{\sqrt{2}}, \\ F_2(x), & x \geq \frac{1}{\sqrt{2}}. \end{cases}$$

(d) Solving for the Global Minimizer (x^*, y^*)

We minimize both cases and pick the smaller objective value.

Case 1: $0 \leq x < \frac{1}{\sqrt{2}}$

$$F_1(x) = (x - 2)^2 + 9.$$

The unconstrained minimizer is $x = 2$, but 2 is not in the interval $[0, \frac{1}{\sqrt{2}})$.

Hence within the domain, the minimum occurs at the boundary:

$$x = \frac{1}{\sqrt{2}}, \quad y = 0.$$

$$F_1\left(\frac{1}{\sqrt{2}}\right) = \left(\frac{1}{\sqrt{2}} - 2\right)^2 + 9 \approx 10.673.$$

Case 2: $x \geq \frac{1}{\sqrt{2}}$

We minimize

$$F_2(x) = (x - 2)^2 + (x^2 - 3.5)^2.$$

Compute the derivative:

$$F_2'(x) = 2(x - 2) + 2(x^2 - 3.5)(2x) = 2(x - 2) + 4x(x^2 - 3.5).$$

Set $F_2'(x) = 0$:

$$2(x - 2) + 4x(x^2 - 3.5) = 0.$$

Simplifying,

$$4x^3 - 12x - 4 = 0 \implies x^3 - 3x - 1 = 0.$$

This cubic has one real solution:

$$x^* \approx 1.879385.$$

Since $1.879 > 1/\sqrt{2} \approx 0.707$, the solution is valid for this regime.
Follower's response:

$$y^* = x^{*2} - \frac{1}{2} \approx (1.879)^2 - 0.5 \approx 3.03.$$

Objective value:

$$F(x^*, y^*) = (1.879 - 2)^2 + (3.03 - 3)^2 \approx 0.0146 + 0.0009 \approx 0.0155,$$

which is much smaller than the value from Case 1.

$$(x^*, y^*) \approx (1.879, 3.03).$$

(e) Regularity and Validity of Reduction

Convexity of the follower problem. For fixed x , the follower objective is

$$f(x, y) = (y - x^2)^2 + y,$$

a quadratic with coefficient $2 > 0$ on y^2 . Thus $f(x, \cdot)$ is **strictly convex** in y , and the feasible set $y \geq 0$ is convex.

Uniqueness of $y^*(x)$. A strictly convex function on a convex domain has a unique minimizer. Thus $y^*(x)$ is single-valued for all x .

Validity of substituting $y^*(x)$ into the upper-level problem. Because the follower solution is unique and varies continuously with x , the bilevel problem can be reduced safely to a single-level problem by replacing y with $y^*(x)$. No pathological behavior (non-uniqueness, discontinuity, or non-convex feasible region) occurs.

Question 4 — Multi-Objective Optimization: Pareto Front

(a) Pareto Optimality

A point (x^*, y^*) is called *Pareto optimal* if there is no other point (x, y) such that

$$f_1(x, y) \leq f_1(x^*, y^*), \quad f_2(x, y) \leq f_2(x^*, y^*),$$

with at least one inequality strict. In other words, no objective can be improved without worsening the other. The set of all such points forms the *Pareto set* and their image is the *Pareto front*.

(b) Weighted-Sum Scalarization

For a weight $w \in [0, 1]$, consider

$$F_w(x, y) = wf_1(x, y) + (1 - w)f_2(x, y).$$

Since both objectives are strictly convex quadratics, F_w is strictly convex and has a unique global minimizer for each w .

(b1) Minimizer $(x(w), y(w))$

Expand:

$$F_w(x, y) = w[(x - 2)^2 + y^2] + (1 - w)[x^2 + (y - 3)^2].$$

Expanding and collecting terms:

$$F_w(x, y) = x^2 - 4wx + y^2 - 6(1 - w)y + \text{const.}$$

Set gradients to zero:

$$\frac{\partial F_w}{\partial x} = 2x - 4w = 0 \quad \Rightarrow \quad x(w) = 2w,$$

$$\frac{\partial F_w}{\partial y} = 2y - 6(1 - w) = 0 \quad \Rightarrow \quad y(w) = 3(1 - w).$$

Thus the weighted-sum minimizers are the parametric line

$$(x(w), y(w)) = (2w, 3(1 - w)), \quad w \in [0, 1].$$

(b2) Pareto Set via Weighted Sum

For convex objectives, every solution of the weighted-sum problem is Pareto optimal. As w varies continuously from 0 to 1, the points $(x(w), y(w))$ sweep the entire convex Pareto set of this problem.

(c) Parametric Expression for the Pareto Front

Evaluate the objectives at the parametric solution:

$$f_1(w) = (2w - 2)^2 + [3(1 - w)]^2 = 4(1 - w)^2 + 9(1 - w)^2 = 13(1 - w)^2,$$

$$f_2(w) = (2w)^2 + (3(1 - w) - 3)^2 = 4w^2 + 9w^2 = 13w^2.$$

Thus the Pareto front is

$$P = \{(13(1 - w)^2, 13w^2) : w \in [0, 1]\}.$$

Let $u = f_1$, $v = f_2$:

$$u = 13(1 - w)^2, \quad v = 13w^2.$$

Solve for w :

$$w = \sqrt{\frac{v}{13}}, \quad 1 - w = \sqrt{\frac{u}{13}}.$$

Eliminating w gives the closed-front relation:

$$\sqrt{u} + \sqrt{v} = \sqrt{13}.$$

(d) Epsilon-Constraint and KKT Connection

Consider the ε -constraint formulation:

$$\min f_1(x, y) \quad \text{s.t.} \quad f_2(x, y) \leq t.$$

The Lagrangian is

$$L(x, y, \lambda) = f_1(x, y) + \lambda(f_2(x, y) - t), \quad \lambda \geq 0.$$

The KKT stationarity condition is

$$\nabla f_1(x, y) + \lambda \nabla f_2(x, y) = 0.$$

Dividing both sides by $(1 + \lambda)$ yields the relation

$$\nabla(w f_1 + (1 - w) f_2) = 0, \quad w = \frac{1}{1 + \lambda}.$$

Thus each value of λ corresponds to a weight w , and the ε -constraint solutions coincide with the weighted-sum Pareto points.

w	$x(w)$	$y(w)$	$f_1(x(w), y(w))$	$f_2(x(w), y(w))$
0	0.0000	3.0000	13.0000	0.0000
0.25	0.5000	2.2500	7.3125	0.8125
0.50	1.0000	1.5000	3.2500	3.2500
0.75	1.5000	0.7500	0.8125	7.3125
1	2.0000	0.0000	0.0000	13.0000

Table 1: Pareto-optimal solutions and objective values for selected weights.

(e) Numerical Values for Selected Weights

Using

$$\begin{aligned} x(w) &= 2w, & y(w) &= 3(1 - w), \\ f_1(w) &= 13(1 - w)^2, & f_2(w) &= 13w^2, \end{aligned}$$

we evaluate at $w \in \{0, 0.25, 0.5, 0.75, 1\}$.

Question 5 — Genetic Algorithm: Jigsaw Puzzle Solver

In this question, we implemented a Genetic Algorithm (GA) capable of assembling a jigsaw puzzle from a natural image. The puzzle pieces are generated by dividing the image into an $r \times c$ grid (e.g., 9×9). Each chromosome represents a full puzzle arrangement, encoded as a permutation of piece indices.

(a) Fit Score Between Two Pieces

To measure how well two puzzle pieces match along their shared boundary, we extract from each piece the pixel arrays of its four edges (top, bottom, left, right). For two touching edges E_A and E_B , we define their compatibility as the negative mean squared difference:

$$\text{Fit}(A, B) = -\frac{1}{N} \sum_{i=1}^N (E_A(i) - E_B(i))^2.$$

A smaller pixel mismatch produces a larger (less negative) score, meaning a better match. Since this computation is repeated many times, we precompute a lookup table containing all pairwise horizontal and vertical edge compatibilities between pieces.

(b) Fitness of a Full Puzzle Arrangement

A candidate solution is a permutation of all puzzle pieces, reshaped into an $r \times c$ grid. The overall fitness of an arrangement is the sum of all valid adjacencies:

$$F(P) = \sum_{i=1}^r \sum_{j=1}^{c-1} \text{Fit}(P_{i,j}, P_{i,j+1}) + \sum_{i=1}^{r-1} \sum_{j=1}^c \text{Fit}(P_{i,j}, P_{i+1,j}),$$

where the first sum accounts for horizontal neighbors and the second for vertical neighbors. Higher scores correspond to more consistent assemblies.

(c) Survival of the Fittest (Selection Mechanism)

To ensure that better puzzle arrangements are more likely to contribute to the next generation, we employ **tournament selection**:

1. Randomly sample $k = 3$ individuals from the population.
2. Select the one with the highest fitness as a parent.

Tournament selection is stable, maintains diversity, and works well with rugged fitness landscapes typical of puzzle problems.

(d) Evolutionary Engine

The complete GA consists of the following components:

Initialization We generate a population of random permutations, each representing a full puzzle configuration.

Crossover Instead of classical permutation crossovers (e.g., PMX), which fail to preserve 2D adjacency, we designed a **kernel-based crossover**:

- Select a random “root piece” and place it in a random cell.
- Grow a *kernel* by expanding to neighboring cells.
- For each frontier position:
 - If both parents place the same piece in that cell, use it.
 - Otherwise, choose from unused pieces the one whose edge best matches the already-placed neighbor (according to the fit score).

This crossover always yields a valid full puzzle arrangement, avoids duplicates, and preserves local spatial structure.

Mutation To maintain diversity, we apply **swap mutation** with probability p_m (typically 0.10–0.15), exchanging two randomly chosen pieces in the chromosome.

Elitism and Replacement At every generation, the top-performing individuals (elite) are copied unchanged into the next population. The remaining positions are filled with offspring produced by selection, kernel crossover, and mutation.

Termination The evolution continues for a fixed number of generations or until the best fitness stabilizes.

Question 7 — Batch vs. Stochastic Gradient Descent

(a) Computational Load per Iteration

Batch Gradient Descent (BGD). BGD computes the gradient using all n samples:

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w).$$

Each iteration is computationally expensive, scaling as $\mathcal{O}(n)$, but produces an exact gradient.

Stochastic Gradient Descent (SGD). SGD updates using only a single data point:

$$\nabla f(w) \approx \nabla f_i(w).$$

Each iteration costs $\mathcal{O}(1)$, making individual steps much faster and enabling quick progress, especially on large datasets.

(b) Noise (Variance) in the Gradient Estimation

BGD. Provides an exact gradient with:

$$\text{Var}[\nabla f(w)] = 0.$$

Thus, updates are smooth and deterministic.

SGD. Uses a noisy estimate of the gradient:

$$\text{Var}[\nabla f_i(w)] > 0.$$

The noise comes from randomness in sampling and causes fluctuation in the update direction. This noise can help escape shallow local minima or plateaus but also prevents clean convergence without step-size scheduling.

(c) Stability and Nature of Convergence

BGD. Stable and converges smoothly toward the global optimum (for convex functions). However, it may require many costly iterations, and each step becomes increasingly slow as data size grows.

SGD. Converges faster in terms of wall-clock time, but convergence is noisy due to gradient variance. With diminishing learning rates (e.g., $\eta_t = \frac{1}{t}$), SGD converges to the exact minimum for convex problems. With constant step size, SGD fluctuates around the optimum in a small neighborhood instead of settling exactly.

Summary

BGD provides stable but computationally expensive updates, using exact gradients each iteration. SGD performs cheap updates and scales better to large datasets, but introduces noise that affects convergence behavior. In practice, SGD or mini-batch SGD is often preferred due to efficiency and strong empirical performance.

Question 8 — Steepest Descent Direction via First-Order Taylor Approximation

We show that the negative gradient direction

$$p_k = -\nabla f(x_k)$$

is the direction of *steepest descent* in a local neighborhood of x_k , under the constraint that the search direction has unit norm.

Proof

Consider the first-order Taylor approximation of f around x_k :

$$f(x_k + \alpha p) \approx f(x_k) + \alpha \nabla f(x_k)^\top p, \quad \alpha > 0 \text{ small.}$$

Since $f(x_k)$ is constant with respect to p , minimizing the expression is equivalent to minimizing

$$\nabla f(x_k)^\top p$$

over all unit vectors $\|p\| = 1$.

Optimization problem. We therefore solve

$$\min_{\|p\|=1} \nabla f(x_k)^\top p.$$

Using the Cauchy–Schwarz inequality. For any vectors a and b ,

$$a^\top b \geq -\|a\|\|b\|,$$

with equality if and only if $b = -a/\|a\|$.

Applying this to $a = \nabla f(x_k)$ and $b = p$, with $\|p\| = 1$, gives

$$\nabla f(x_k)^\top p \geq -\|\nabla f(x_k)\|.$$

Minimum value and minimizer. The minimum possible value is

$$\min_{\|p\|=1} \nabla f(x_k)^\top p = -\|\nabla f(x_k)\|,$$

and it is achieved precisely when

$$p^* = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

Thus the steepest descent *unit direction* is the normalized negative gradient.

Corresponding unnormalized direction. Gradient descent uses the direction

$$p_k = -\nabla f(x_k),$$

which is simply the steepest unit descent direction scaled by $\|\nabla f(x_k)\|$. Scaling does not change direction, only step length, and therefore p_k is the direction of steepest decrease in the first-order approximation sense.

Conclusion

By minimizing the first-order Taylor approximation of $f(x_k + \alpha p)$ over all unit directions $\|p\| = 1$, we show that the direction of steepest descent is the normalized negative gradient. Thus the gradient descent update direction,

$$p_k = -\nabla f(x_k),$$

is indeed the steepest descent direction in a local neighborhood of x_k .

Question 9 — Pseudoinverse Solution as the Least-Squares Minimizer

Consider an overdetermined linear system

$$Ax = b, \quad A \in \mathbb{R}^{m \times n}, \quad m > n,$$

where A has full column rank (i.e., $\text{rank}(A) = n$). We define the least-squares loss

$$L(x) = \|Ax - b\|_2^2.$$

We show that the pseudoinverse solution

$$x^* = (A^\top A)^{-1} A^\top b$$

is the unique global minimizer of $L(x)$.

Proof

1. Expand the objective. We write $L(x)$ as

$$L(x) = (Ax - b)^\top (Ax - b) = x^\top A^\top Ax - 2b^\top Ax + b^\top b.$$

2. Compute the gradient. Taking the gradient with respect to x ,

$$\nabla L(x) = 2A^\top Ax - 2A^\top b.$$

3. Set the gradient to zero. Stationary points satisfy:

$$A^\top Ax = A^\top b,$$

which are the *normal equations*. Since A has full column rank, $A^\top A$ is symmetric positive definite and therefore invertible. Thus the unique solution of the normal equations is

$$x^* = (A^\top A)^{-1} A^\top b.$$

4. Show x^* is the global minimizer. Because $A^\top A$ is positive definite, the quadratic function

$$L(x) = x^\top A^\top A x - 2b^\top A x + b^\top b$$

is strictly convex. A strictly convex function has at most one stationary point, and that point is its unique global minimizer.

Since x^* is the unique stationary point of $L(x)$, it follows that

$$x^* = \arg \min_x \|Ax - b\|_2^2.$$

Question 10 — Newton and Quasi-Newton Methods

Newton's method is a second-order optimization technique that uses curvature information through the Hessian. Quasi-Newton methods improve its computational efficiency by avoiding explicit Hessian construction and inversion.

(a) Newton Step Computation

Given a twice-differentiable objective $f : \mathbb{R}^n \rightarrow \mathbb{R}$, Newton's method updates

$$x_{k+1} = x_k + p_k,$$

where the Newton step p_k solves

$$\nabla^2 f(x_k) p_k = -\nabla f(x_k).$$

Equivalently,

$$p_k = -(\nabla^2 f(x_k))^{-1} \nabla f(x_k).$$

This step uses second-order information to make a locally quadratic approximation of f .

(b) When Newton's Method Can Fail to Converge

Newton's method may fail to converge to a minimum in several cases:

- The Hessian $\nabla^2 f(x_k)$ is not positive definite, so the step may point toward a saddle point or maximum.

- If the starting point is far from a local minimum, the quadratic approximation may be poor, causing divergence.
- Newton steps can overshoot if the Hessian is ill-conditioned or if the function is not well approximated by a quadratic locally.

(c) When the Newton Direction is Not a Descent Direction

The Newton direction p_k is a descent direction if and only if

$$\nabla f(x_k)^\top p_k < 0.$$

Using the Newton equation, this becomes

$$-\nabla f(x_k)^\top (\nabla^2 f(x_k))^{-1} \nabla f(x_k) < 0.$$

The expression is negative only when $\nabla^2 f(x_k)$ is *positive definite*. Thus:

- If $\nabla^2 f(x_k)$ is indefinite or negative definite, the Newton direction is not a descent direction.
- In such cases, Newton's method may move uphill or toward a saddle point.

(d) Main Idea of Quasi-Newton Methods

Quasi-Newton methods aim to retain the fast local convergence of Newton's method while avoiding:

- the expensive computation of the full Hessian,
- the $\mathcal{O}(n^3)$ cost of Hessian inversion.

Their main idea is to iteratively build an approximation of the Hessian (or its inverse) using only gradient information and low-rank matrix updates.

(e) Approximating the Hessian via the Secant Equation

Let

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

The *secant equation* imposes the quasi-Newton condition:

$$B_{k+1}s_k = y_k,$$

mirroring the ideal Hessian update from the mean value theorem. Algorithms like BFGS and DFP construct B_{k+1} (or its inverse H_{k+1}) as a rank-one or rank-two update of B_k , ensuring:

- symmetry,
- positive definiteness (under mild conditions),
- curvature consistency with s_k and y_k .

Thus, quasi-Newton methods achieve second-order behavior while avoiding explicit second derivative calculations and costly Hessian inversions.

Question 11 — Quadratic Convergence of Newton's Method

(a) Definition of Quadratic Convergence

Let x^* be the minimizer of f and define the error

$$e_k = x_k - x^*.$$

A sequence $\{x_k\}$ converges *quadratically* to x^* if

$$\|e_{k+1}\| \leq C\|e_k\|^2 \quad \text{for some constant } C > 0.$$

Quadratic convergence means that the number of correct digits roughly doubles at each iteration.

Contrast with linear convergence. A sequence converges *linearly* if

$$\|e_{k+1}\| \leq \rho\|e_k\| \quad \text{with } 0 < \rho < 1,$$

i.e., the error decreases proportionally to the previous error. Newton's method, under suitable assumptions, is much faster because the error decreases proportionally to its square.

(b) Newton Step Approximates $x^* - x_k$ Near the Minimizer

Let x^* be a local minimizer, so

$$\nabla f(x^*) = 0, \quad \nabla^2 f(x^*) \succ 0.$$

Taylor expansion of the gradient. Expand $\nabla f(x)$ around x^* :

$$\nabla f(x_k) = \nabla f(x^*) + \nabla^2 f(x^*)(x_k - x^*) + R_k,$$

where the remainder satisfies

$$\|R_k\| = \mathcal{O}(\|x_k - x^*\|^2).$$

Using $\nabla f(x^*) = 0$ and $e_k = x_k - x^*$, this becomes

$$\nabla f(x_k) = \nabla^2 f(x^*)e_k + R_k.$$

Newton direction. The Newton step is defined by

$$p_k = -(\nabla^2 f(x_k))^{-1} \nabla f(x_k).$$

For x_k sufficiently close to x^* , the Hessian satisfies

$$\nabla^2 f(x_k) = \nabla^2 f(x^*) + \mathcal{O}(\|e_k\|),$$

so to first order we may replace $\nabla^2 f(x_k)$ with $\nabla^2 f(x^*)$. Substituting the Taylor expansion of the gradient gives:

$$p_k \approx -(\nabla^2 f(x^*))^{-1} (\nabla^2 f(x^*)e_k + R_k) = -e_k - (\nabla^2 f(x^*))^{-1} R_k.$$

Because $R_k = \mathcal{O}(\|e_k\|^2)$, we have

$$p_k = -(x_k - x^*) + \mathcal{O}(\|e_k\|^2).$$

Newton update. The new iterate is

$$x_{k+1} = x_k + p_k \approx x_k - (x_k - x^*) + \mathcal{O}(\|e_k\|^2) = x^* + \mathcal{O}(\|e_k\|^2).$$

Thus the new error satisfies

$$e_{k+1} = x_{k+1} - x^* = \mathcal{O}(\|e_k\|^2),$$

which is exactly the definition of quadratic convergence.

Question 12 — Rosenbrock Function: Gradient Descent vs. Newton's Method

We study the Rosenbrock function

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2,$$

whose global minimizer is $x^* = (1, 1)$ with $f(x^*) = 0$. We derive the gradient and Hessian, implement Gradient Descent (GD) and Newton's Method, and compare their convergence behavior.

Gradient and Hessian

Let $x = (x_1, x_2)^\top$. The gradient is:

$$\nabla f(x) = \begin{bmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{bmatrix}.$$

The Hessian is:

$$H(x) = \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 \\ -400x_1 & 200 \end{bmatrix}.$$

Algorithms

Steepest Descent (Gradient Descent).

$$x_{k+1} = x_k - \alpha \nabla f(x_k), \quad \alpha = 0.001.$$

Newton's Method.

$$x_{k+1} = x_k - H(x_k)^{-1} \nabla f(x_k), \quad \alpha = 1.$$

Results: Trajectories and Convergence

Trajectory Visualization

Figure 1 shows the optimization paths overlaid on contour lines of the Rosenbrock function.

Key observations:

- **Gradient Descent** (red) takes very small steps and moves slowly along the curved valley.

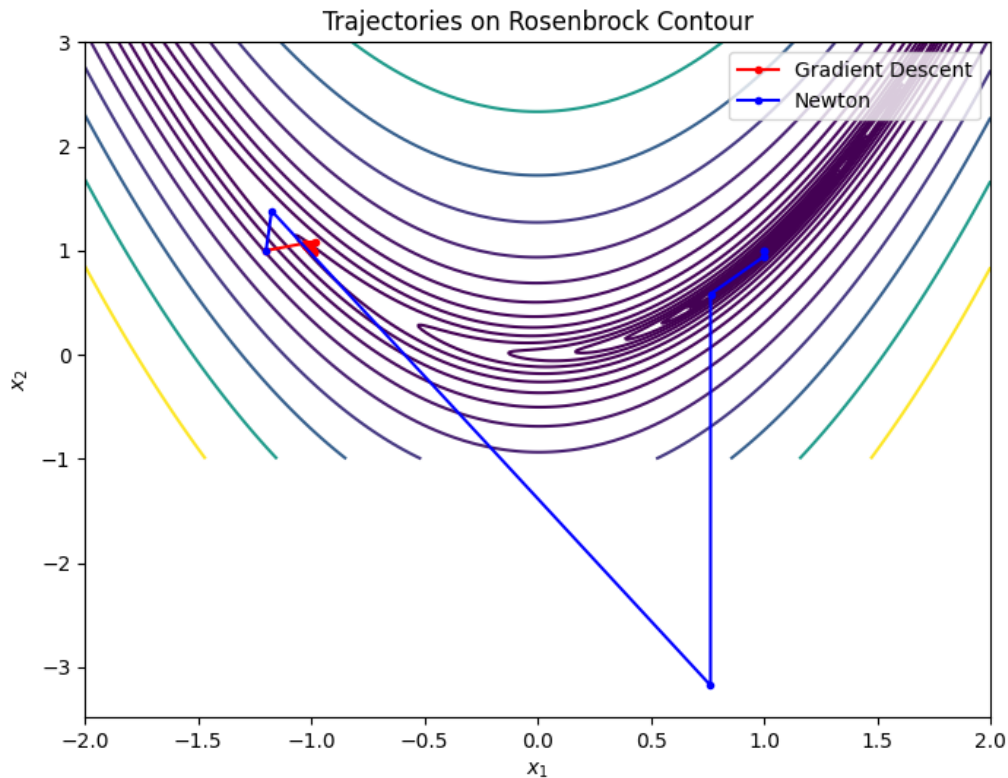


Figure 1: Trajectories of Gradient Descent and Newton's Method on the Rosenbrock function.

- **Newton's Method** (blue) makes large jumps initially, overshooting the valley. This behavior is expected when the method is applied far from the minimizer with full step size.
- Once Newton's trajectory reaches the valley, it rapidly converges toward (1, 1).

Convergence Plot

Figure 2 shows the values of $\log_{10} f(x_k)$.

Interpretation:

- **Gradient Descent:** The function value decreases slowly and almost linearly in the log scale, characteristic of linear convergence.
- **Newton:** After the initial unstable iterations, $f(x_k)$ drops extremely rapidly. The near-vertical slope in the log plot clearly indicates *quadratic convergence*.
- The final Newton values reach machine precision within a few iterations.

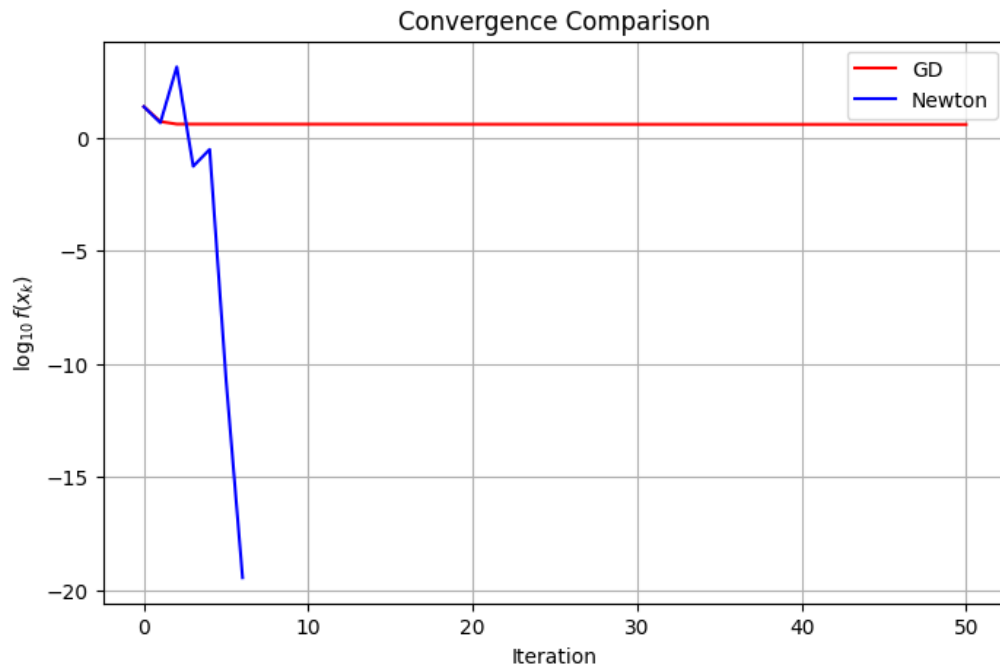


Figure 2: Convergence comparison in terms of $\log_{10} f(x_k)$ for Gradient Descent and Newton's Method.

Summary

The Rosenbrock function highlights the contrast between first-order and second-order methods:

- Gradient Descent progresses slowly in the presence of narrow, curved valleys.
- Newton's Method is highly sensitive to initialization but exhibits superior local convergence.

These results validate the theoretical understanding of linear vs. quadratic convergence rates.

Question 13 — Equality-Constrained Optimization via Lagrange Multipliers and Newton–Raphson

We solve the constrained optimization problem

$$\min_{x,y} f(x,y) = e^{x^2+y^2} \quad \text{s.t.} \quad g(x,y) = x + y - 1 = 0.$$

This constraint describes a straight line, while the objective grows radially like e^{r^2} , so the solution lies at the point on the line closest to the origin.

(a) Lagrangian

The Lagrangian of the constrained problem is

$$L(x, y, \lambda) = e^{x^2+y^2} + \lambda(x + y - 1).$$

(b) Solve $\nabla L = 0$ for (x^*, y^*, λ^*)

Compute the partial derivatives:

$$\begin{aligned}\frac{\partial L}{\partial x} &= 2xe^{x^2+y^2} + \lambda = 0, \\ \frac{\partial L}{\partial y} &= 2ye^{x^2+y^2} + \lambda = 0, \\ \frac{\partial L}{\partial \lambda} &= x + y - 1 = 0.\end{aligned}$$

Subtract the first two equations:

$$2xe^{x^2+y^2} = 2ye^{x^2+y^2} \quad \Rightarrow \quad x = y.$$

Using the constraint:

$$x + y = 1 \quad \Rightarrow \quad x = y = \frac{1}{2}.$$

Plug this into the first Lagrange equation:

$$2\left(\frac{1}{2}\right)e^{1/2} + \lambda = 0 \quad \Rightarrow \quad \lambda^* = -e^{1/2}.$$

Thus the optimal point is:

$$x^* = y^* = \frac{1}{2}, \quad \lambda^* = -e^{1/2}.$$

(c) The Nonlinear System $F(w) = 0$

Define

$$w = \begin{bmatrix} x \\ y \\ \lambda \end{bmatrix}, \quad F(w) = \begin{bmatrix} 2xe^{x^2+y^2} + \lambda \\ 2ye^{x^2+y^2} + \lambda \\ x + y - 1 \end{bmatrix}.$$

We seek $F(w) = 0$.

(d) Jacobian $JF(w)$

Compute the Jacobian matrix:

$$JF(w) = \begin{bmatrix} \frac{\partial F_1}{\partial x} & \frac{\partial F_1}{\partial y} & \frac{\partial F_1}{\partial \lambda} \\ \frac{\partial F_2}{\partial x} & \frac{\partial F_2}{\partial y} & \frac{\partial F_2}{\partial \lambda} \\ \frac{\partial F_3}{\partial x} & \frac{\partial F_3}{\partial y} & \frac{\partial F_3}{\partial \lambda} \end{bmatrix}.$$

Compute term by term. Let $S = e^{x^2+y^2}$.

$$\frac{\partial F_1}{\partial x} = 2S + 4x^2S, \quad \frac{\partial F_1}{\partial y} = 4xyS, \quad \frac{\partial F_1}{\partial \lambda} = 1.$$

$$\frac{\partial F_2}{\partial x} = 4xyS, \quad \frac{\partial F_2}{\partial y} = 2S + 4y^2S, \quad \frac{\partial F_2}{\partial \lambda} = 1.$$

$$\frac{\partial F_3}{\partial x} = 1, \quad \frac{\partial F_3}{\partial y} = 1, \quad \frac{\partial F_3}{\partial \lambda} = 0.$$

Thus,

$$JF(w) = \begin{bmatrix} 2S(1 + 2x^2) & 4xyS & 1 \\ 4xyS & 2S(1 + 2y^2) & 1 \\ 1 & 1 & 0 \end{bmatrix}.$$

(e) Newton–Raphson Implementation and Convergence

We apply Newton iterations:

$$w_{k+1} = w_k - JF(w_k)^{-1}F(w_k), \quad w_0 = (0, 0, 0)^\top.$$

The method converges rapidly to the analytic solution:

$$w^* = \begin{bmatrix} 1/2 \\ 1/2 \\ -e^{1/2} \end{bmatrix}.$$

Empirical results show that only a few iterations are required to reach machine-precision accuracy.

Question 14 — Step-Size Strategies in Gradient Descent

We compare two different step-size selection strategies applied to Gradient Descent (GD):

- Backtracking line search using the Armijo condition.
- An adaptive method (AdaGrad).

Both are evaluated on the Rosenbrock function from Question 12, starting from

$$x_0 = (-1.2, 1)^\top.$$

(a) Step-Size Selection Methods

Backtracking Line Search (Armijo Rule). Starting from an initial step size α_0 , the step size is repeatedly shrunk by a factor $\beta \in (0, 1)$ until the Armijo condition holds:

$$f(x_k - \alpha \nabla f(x_k)) \leq f(x_k) - c\alpha \|\nabla f(x_k)\|^2$$

where $c \in (0, 1)$ is a small constant. This ensures sufficient decrease and avoids overshooting, making GD more robust on ill-conditioned problems.

AdaGrad (Adaptive Gradient Method). AdaGrad adapts the step size per coordinate based on historical squared gradients:

$$x_{k+1} = x_k - \frac{\eta}{\sqrt{G_k} + \epsilon} \odot \nabla f(x_k), \quad G_k = \sum_{i=1}^k \nabla f(x_i) \odot \nabla f(x_i),$$

where $\odot$ denotes elementwise multiplication. Large gradients reduce future step sizes, stabilizing updates.

(b) Gradient Descent Implementations

We implement:

1. GD with backtracking line search.
2. GD with AdaGrad.

Both are run for at most 2000 iterations or until $\|\nabla f(x_k)\| < 10^{-6}$.

(c) Comparison of Results

Figure 3 shows the optimization trajectories on the Rosenbrock contour plot.

Figure 4 shows $\log_{10} f(x_k)$ versus iteration.

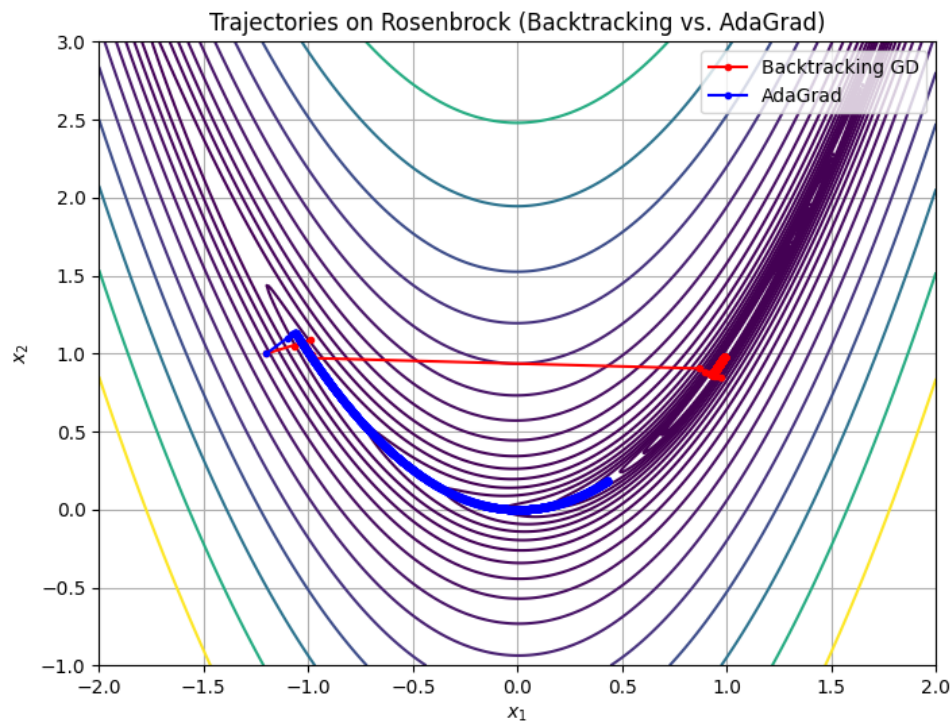


Figure 3: Trajectories of GD with Backtracking and AdaGrad on the Rosenbrock function.

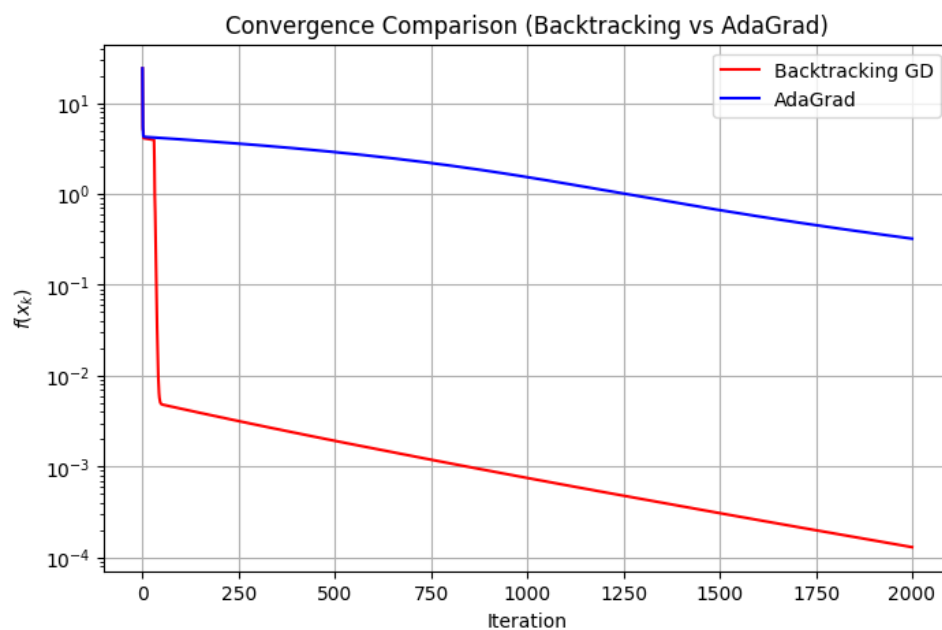


Figure 4: Convergence comparison of Backtracking GD and AdaGrad.

Observations:

- **Backtracking GD** significantly outperforms fixed-step GD from Question 12. It adapts the step size to account for curvature, making progress even in the narrow Rosenbrock valley.

- **AdaGrad** stabilizes updates early in the valley but slows down afterward due to accumulated gradient magnitudes, leading to diminishing step sizes.
- **Trajectory Comparison:** Backtracking GD follows the valley efficiently. AdaGrad takes smoother but increasingly smaller steps.
- **Convergence Rate:** Backtracking GD achieves near-linear convergence and reaches the minimizer first. AdaGrad converges reliably but more slowly once the accumulated gradient sum becomes large.

Summary

Adaptive step-size methods dramatically improve stability compared to fixed-step GD. Backtracking dynamically adjusts to the Rosenbrock curvature, while AdaGrad stabilizes the search at the cost of slower long-term progress.

References

Include any sources you used, such as websites, articles, or textbooks.